

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..25.4

CUSTOM OVERRIDES

 Copy as Markdown

Custom authentication overrides

INFO

This page documents the legacy `serverpod_auth` module. To move an existing app to the current authentication framework, see [Migrate from legacy auth](#).

If the legacy `serverpod_auth` module does not fulfill your requirements, you can implement your own authentication handling. Serverpod is designed to make it easy to add custom authentication overrides.

Server setup

When running a custom auth integration it is up to you to build the authentication model and issuing auth tokens.

Token validation

The token validation is performed by providing a custom `AuthenticationHandler` callback when initializing Serverpod. The callback should return an `AuthenticationInfo` object if the token is valid, otherwise `null`.

```
final pod = Serverpod(  
  args,  
  authenticationHandler: (Session session, String token) async {  
    /// Custom validation handler  
    if (token != 'valid') return null;  
  
    return AuthenticationInfo('1', <Scope>{}, authId: 'valid');  
  },  
);
```



Ask AI

In the above example, the `authenticationHandler` callback is overridden with a custom validation method. The method returns an `AuthenticationInfo` object with `userIdentifier` `"1"`, no scopes, and `authId` `"valid"` if the token is the literal `"valid"`, otherwise `null`.

NOTE

The `userIdentifier` passed to the `AuthenticationInfo` constructor, as the first parameter, is a `String` and must not be empty. Since the default implementation of `serverpod_auth` uses numeric IDs for the users, the legacy module provides a convenience getter `userId`, which parses the identifier and returns the integer value.

NOTE

In the `authenticationHandler` callback the `authenticated` field on the session will always be `null` as it is the `authenticationHandler` that figures out who the user is.

INFO

The required `authId` field in the `AuthenticationInfo` object links the user to a specific authentication id, for example a device session. It is what `RevokedAuthenticationAuthId` matches against when revoking authentication for a specific device.

Scopes

The scopes returned from the `authenticationHandler` is used to grant access to scope restricted endpoints. The `Scope` class is a simple wrapper around a nullable `String` in dart. This means that you can format your scopes however you want as long as they are in a `String` format.

Normally if you implement a JWT you would store the scopes inside the token. When extracting them all you have to do is convert the `String` stored in the token into a `Scope` object by calling the constructor.

```
List<String> scopes = extractScopes(token);
Set<Scope> userScopes = scopes.map((scope) => Scope(scope)).toSet();
```

Handling revoked authentication

When a user's authentication is revoked, the server must be notified to respect the changes (e.g. to close method streams). Invoke the `session.messages.authenticationRevoked` method and raise the appropriate event to notify the server.

```
var userIdentifier = '1';
var revokedScopes = ['write'];
```

```

var message = RevokedAuthenticationScope(
    scopes: revokedScopes,
);

await session.messages.authenticationRevoked(
    userIdentifier,
    message,
);

```

Parameters

- `userIdentifier` - The `userIdentifier` of the `AuthenticationInfo` object to be revoked.
- `message` - The revoked authentication event message. See below for the different type of messages.

Revoked authentication messages

There are three types of `RevokedAuthentication` messages that are used to specify the extent of the authentication revocation:

Message type	Description
<code>RevokedAuthenticationUser</code>	All authentication is revoked for a user.
<code>RevokedAuthenticationAuthId</code>	A single authentication id is revoked for the user. This should match the <code>authId</code> field in the <code>AuthenticationInfo</code> object.
<code>RevokedAuthenticationScope</code>	List of scopes that have been revoked for a user.

Each message type provides a tailored approach to revoke authentication based on different needs.

Send token to client

You are responsible for implementing the endpoints to authenticate/authorize the user. But as an example such an endpoint could look like the following.

```

class UserEndpoint extends Endpoint {
    Future<LoginResponse?> login(
        Session session,
        String username,
        String password,
    ) async {
        var identifier = authenticateUser(session, username, password);
        if (identifier == null) return null;
    }
}

```

```

        return issueMyToken(identifier, scopes: {});
    }
}

```

In the above example, the `login` method authenticates the user and creates an auth token. The token is then returned to the client.

Client setup

Enabling authentication in the client is as simple as configuring a key manager and placing any token in it. If a key manager is configured, the client will automatically query the manager for a token and include it in communication with the server.

Configure key manager

Key managers need to implement the `AuthenticationKeyManager` interface. The interface's abstract `toHeaderValue` method formats the token for the HTTP authorization header. The class is deprecated in favor of `ClientAuthKeyProvider`, so the analyzer warns on it. It keeps working with the legacy module. The key manager is assigned to the client's `authKeyProvider` field. If no key manager is configured, the client will not include tokens in requests to the server.

```

class SimpleAuthKeyManager extends AuthenticationKeyManager {
    String? _key;

    @override
    Future<String?> get() async {
        return _key;
    }

    @override
    Future<void> put(String key) async {
        _key = key;
    }

    @override
    Future<void> remove() async {
        _key = null;
    }

    @override
    Future<String?> toHeaderValue(String? key) async {
        if (key == null) return null;
        return wrapAsBasicAuthHeaderValue(key);
    }
}

var keyManager = SimpleAuthKeyManager();

var client = Client('http://localhost:8080/')

```

```
..authKeyProvider = keyManager
..connectivityMonitor = FlutterConnectivityMonitor();
```

In the above example, the `SimpleAuthKeyManager` is configured as the client's authentication key manager. The `SimpleAuthKeyManager` stores the token in memory.

INFO

The `SimpleAuthKeyManager` is not practical and should only be used for testing. A secure implementation of the key manager is available in the `serverpod_auth_shared_flutter` package named `FlutterAuthenticationKeyManager`. It provides safe, persistent storage for the auth token.

The sections below use the `keyManager` reference created above to interact with the stored token.

Store token

When the client receives a token from the server, it is responsible for storing it in the key manager using the `put` method. The key manager will then include the token in all requests to the server.

```
await keyManager.put(token);
```

In the above example, the `token` is placed in the key manager. It will now be included in communication with the server.

Remove token

To remove the token from the key manager, call the `remove` method.

```
await keyManager.remove();
```

The above example removes any token from the key manager.

Retrieve token

To retrieve the token from the key manager, call the `get` method.

```
var token = await keyManager.get();
```

The above example retrieves the token from the key manager and stores it in the `token` variable.

Authentication schemes

The module's shipped key manager, `FlutterAuthenticationKeyManager`, passes the authentication token from client to server in accordance with the HTTP `authorization` header standard, with the `basic` scheme name and encoding. This is securely transferred as the connection is TLS encrypted.

The legacy module's `FlutterAuthenticationKeyManager` encodes and wraps the user-provided token in a `basic` scheme with the `wrapAsBasicAuthHeaderValue` helper. The value is automatically unwrapped on the server side before being handed to the user-provided authentication handler described above.

For apps using `FlutterAuthenticationKeyManager`, this wrapping and unwrapping is invisible to application code.

Implementing your own authentication scheme

If you are implementing your own authentication and are using the `basic` scheme, note that this is supported but will be automatically unwrapped i.e. decoded on the server side before being handed to your `AuthenticationHandler` implementation. It will in this case receive the decoded auth key value after the `basic` scheme name.

If you are implementing a different authentication scheme, for example OAuth 2 using bearer tokens, implement the `toHeaderValue` method of your key manager accordingly. This client-side method converts the authentication key to the format that is sent as a transport header to the server.

On the server side, both `basic` and `bearer` values are automatically unwrapped before the `AuthenticationHandler` is invoked. The handler receives the bare token without the scheme prefix. Other schemes are passed through unchanged and need manual parsing in the handler.

The header value must be compliant with the HTTP header format defined in RFC 9110 HTTP Semantics, 11.6.2. Authorization. See:

- [HTTP Authorization header](#)
- [RFC 9110, 11.6.2. Authorization](#)

An approach to adding OAuth handling might make changes to the above code akin to the following.

Client side:

```
class MyOAuthKeyManager extends AuthenticationKeyManager {
  String? _key;

  @override
  Future<String?> get() async {
    return _key;
  }

  @override
  Future<void> put(String key) async {
```

```

    _key = key;
  }

  @override
  Future<void> remove() async {
    _key = null;
  }

  @override
  Future<String?> toHeaderValue(String? key) async {
    if (key == null) return null;
    return 'Bearer ${myBearerTokenObtainer(key)}';
  }
}

var client = Client('http://localhost:8080/')
..authKeyProvider = MyOAuthKeyManager()
..connectivityMonitor = FlutterConnectivityMonitor();

```

Server side:

```

final pod = Serverpod(
  args,
  authenticationHandler: (Session session, String token) async {
    /// Bearer token validation handler
    var (uid, scopes, authId) = myBearerTokenValidator(token);
    if (uid == null) return null;

    return AuthenticationInfo(uid, scopes, authId: authId);
  },
);

```